National University of Singapore
School of Computing

CP4101 B.Comp. Dissertation — Continuous Assessment (CA) Report

MorphAPI: Type-Grounded Hybrid AST and LLM Repair for Self-Maintaining API Migrations

[Your Full Name]
Matriculation No.: [A0XXXXXX]

Project No.: [HXXXXXX]
Supervisor: [Supervisor Name]
Main Evaluator: [Evaluator Name]

AY2026/27 Semester 1 — Submitted Week 12

Abstract. Library APIs change constantly, and each breaking change forces client code to be migrated. Rule-based codemods are precise but can only apply transformations someone has written down; large language models (LLMs) can infer new code from vague migration guides but invent symbols that do not exist (for example `CountryCode.US` where the SDK defines `CountryCode.Us`). This project builds MorphAPI, a hybrid migration agent for TypeScript in which static analysis decides *where* to edit and *what is true*, an LLM proposes *how* to rewrite, and the TypeScript type checker decides *what is accepted*. The core artefact so far is a Code Property Graph built on the project's real compiler program: an abstract syntax tree (AST) annotated with resolved types, data flow, call edges and links from each deprecated API symbol to its successor. For each call site the graph produces a compact set of *slice facts* that replaces the whole file in the prompt, and a type-checker gate rejects hallucinated names with the valid alternatives as feedback. On a benchmark of 11 migration scenarios (21 call sites), the full pipeline completed all 11 with gpt-4o-mini and with Claude Sonnet 4.5 and changed no unrelated lines, whereas an LLM-only baseline completed 5 and rewrote 67% unrelated lines. Ablations show that the type-checker gate and one-hop impact repair account for most of the gain. Next semester will scale the evaluation to repeated runs and real open-source migrations.

Contents

1 Introduction and Project Objectives **2**

1.1 Motivation . . . 2

1.2 Problem statement . . . 2

1.3 Objectives . . . 2

1.4 Scope . . . 3

2 Literature Review **3**

2.1 API evolution and automated migration . . . 3

2.2 Program representations . . . 4

2.3 LLMs for code: capabilities and hallucination . . . 4

2.4 Grounding and verifying LLM output . . . 4

2.5 Gap and positioning . . . 4

3 Progress So Far **5**

3.1 Overview . . . 5

3.2 Benchmark (O1) . . . 5

3.3 Loading the project and finding call sites . . . 6

3.4 The Code Property Graph: an AST with metadata (O2) . . . 6

3.5 Slice facts: what the LLM receives . . . 8

3.6 Oracle extraction and the type-checker gate (O3) . . . 9

3.7 Follow-up steps and impact repair (O4) . . . 10

3.8 Design alternatives investigated . . . 10

3.9 Preliminary evaluation (O5) . . . 11

3.10 Tooling for supervision . . . 12

4 Research Plan for Next Semester **12**

4.1 Research questions . . . 12

4.2 Planned work . . . 12

4.3 Timeline . . . 13

4.4 Risks . . . 13

5 Conclusion **13**

1 Introduction and Project Objectives

1.1 Motivation

Software is built on third-party libraries whose APIs evolve. Parameters are renamed, plain strings become enumerations, synchronous calls become asynchronous, results are wrapped in pagination envelopes and authentication schemes are replaced. Empirical studies show that breaking changes are common and that most of them are refactorings that client developers must replicate by hand (Dig and Johnson, 2006; Xavier et al., 2017); developers frequently delay upgrades because the migration cost is high, even when security advisories are involved (Kula et al., 2018; Bogart et al., 2016). Migration is therefore recurring, expensive maintenance work.

Two kinds of tools exist today. *Rule-based* tools such as codemods (Meta Open Source, 2026; Moderne, Inc., 2026) operate on the abstract syntax tree (AST) and are precise, but each transformation must be written by an engineer, and they cannot synthesise code for conceptual changes such as a builder pattern or a JWKS key-set lookup. *Generative* tools based on large language models (LLMs) can infer such rewrites from prose migration guides, but they hallucinate API members that look plausible and do not exist (Liu et al., 2024; Spracklen et al., 2025), rewrite unrelated code, and miss breakage that propagates to callers.

Listing 1 shows the running example used throughout this report. In the old Plaid Link SDK, country codes are strings; in the new SDK they are members of an enum. The public guide only says to "use the country code enum for the United States". The value is `"US"`, but the member *name* is `Us`. Without the SDK's type declarations, an LLM writes `CountryCode.US`, which looks correct and does not compile.

Listing 1: Running example: a Plaid Link call that must move from `plaid-link-v1` (strings) to `plaid-link-v2` (enums).

```
// plaid-link-v2: enum CountryCode { Us = "US", Gb = "GB", Ca = "CA", Fr = "FR", De = "DE" }
const response = await client.linkTokenCreate({
  user: { client_user_id: userId },
  products: ["transactions"],        // v2: Products.Transactions
  country_codes: ["US"],             // v2: CountryCode.Us   (an LLM writes CountryCode.US)
  language: "en",
});
return response.link_token;
```

1.2 Problem statement

Given a TypeScript project that uses a deprecated API, a successor package, and an informal migration guide, produce a minimal edit that (i) migrates every call site, (ii) uses only symbols that exist in the successor API, (iii) repairs breakage that the change causes in neighbouring code, and (iv) leaves unrelated code untouched.

1.3 Objectives

O1 Benchmark. Build a controlled benchmark of API-migration scenarios that each isolate one failure mode observed in practice (enum scaffolding, stale parameters, unit shifts, test evasion, payload envelopes, async contagion, multi-site refactoring, exception hierarchy drift, discriminator changes and authentication overhauls).

O2 Program representation. Design a typed program graph that captures what an LLM needs to know about one call site, beyond what a syntax tree alone provides, and reduce it to a compact prompt context.

O3 Verification. Ground the LLM's proposals in the successor API's real declarations, reject invented symbols and evasion, and return targeted feedback.

O4 Impact. Detect and repair breakage that a migration causes one hop away (callers, result consumers and importers).

O5 Evaluation. Compare the hybrid against an LLM-only baseline and against ablations that remove one mechanism at a time.

1.4 Scope

The project targets TypeScript, because its compiler exposes a full type checker through a public API (Microsoft, 2026; Bierman et al., 2014) and because typed SDKs (Stripe, Plaid, AWS, OpenAI) are widely used. "Correct" is judged at two levels: the type level (the migrated program type-checks with no new diagnostics and no remaining use of the deprecated API) and, in the per-scenario baselines, a behavioural level (for example that amounts were converted from dollars to cents).

2 Literature Review

This review is organised around the three questions the project must answer: how API migration has been automated, how programs can be represented for analysis and for learning, and how LLM code generation can be grounded and verified.

2.1 API evolution and automated migration

Dig and Johnson (2006) studied five Java libraries and found that over 80% of the API changes that broke clients were refactorings, which motivated tools that *record and replay* refactorings: CatchUp! (Henkel and Diwan, 2005) captured refactorings in the library developer's IDE and replayed them on clients. LibSync (Nguyen et al., 2010) learned adaptation patterns from clients that had already migrated, representing API usages as graphs. More recent work infers and applies type changes from version history (Ketkar et al., 2022). A systematic review by Lamothe et al. (2021) notes that most approaches need examples of the migration or a hand-written mapping, and that conceptual changes beyond one-to-one mappings remain hard. Studies of developer behaviour explain why tool support matters: developers react slowly to deprecation (Robbes et al., 2012), and ecosystem norms push the migration cost onto clients (Bogart et al., 2016).

Industrial codemod frameworks (Meta Open Source, 2026; Moderne, Inc., 2026) follow the same rule-based principle. They give precise, reviewable edits, but every pattern has to be authored; a rename recipe cannot decide that `"US"` should become `CountryCode.Us` unless an engineer encodes that mapping.

LLM-based migration is recent. Almeida et al. (2024) report first results for migrating a Python library with ChatGPT: many migrations were correct, but some were incomplete or introduced errors. Nikolov et al. (2025) describe Google's internal use of LLMs for large code migrations, where generated changes are validated by builds and tests and reviewed by humans. Both point to the same pattern that MorphAPI adopts: generation must be paired with automatic validation. Neither work provides the type-grounded, per-call-site context and feedback that this project studies.

2.2 Program representations

A syntax tree records the grammatical structure of one file. Classical program analysis adds semantic relations on top of it. The program dependence graph (Ferrante et al., 1987) combines control and data dependences, and program slicing (Weiser, 1981) extracts the statements relevant to a point of interest. Yamaguchi et al. (2014) merged the AST, control-flow graph and program dependence graph into one *Code Property Graph* (CPG), in which nodes and edges carry key–value properties that can be queried; they used it to find vulnerabilities. MorphAPI adapts this idea to migration: the properties of interest are resolved types, the declaration a call resolves to, and the mapping from deprecated to successor symbols.

Learned representations follow the same intuition that structure beyond tokens matters. Allamanis et al. (2018) add data-flow edges to the AST for graph neural networks. CodeBERT (Feng et al., 2020) pre-trains a transformer on code and text, and GraphCodeBERT (Guo et al., 2021) adds a data-flow graph to pre-training through a graph-guided attention mask. These models produce embeddings that measure similarity; they do not decide whether a symbol exists in a given SDK version, which is the question a migration gate must answer exactly (Section 3.8).

2.3 LLMs for code: capabilities and hallucination

Code LLMs such as Codex (Chen et al., 2021) solve many function-level tasks, but repository-level tasks remain hard: on SWE-bench (Jimenez et al., 2024), models must edit real repositories and pass tests, and early systems resolved only a small fraction of issues. A recurring failure is hallucination. Liu et al. (2024) build a taxonomy of hallucinations in generated code, including references to non-existent APIs and members; Spracklen et al. (2025) show that code LLMs frequently recommend packages that do not exist. API migration is particularly exposed: the model's training data mixes old and new API versions, and migration guides are often vague.

2.4 Grounding and verifying LLM output

Three families of techniques reduce these errors. *Retrieval* puts relevant context into the prompt: retrieval-augmented generation (Lewis et al., 2020) and repository-level retrieval such as RepoCoder (Zhang et al., 2023). Retrieval selects text by similarity; it does not guarantee the facts are complete or type-correct. *Constrained decoding* uses static analysis during generation. Monitor-guided decoding (Agrawal et al., 2023) queries a language server to restrict tokens to members that exist; Repilot (Wei et al., 2023) uses a completion engine to prune infeasible tokens during program repair; and type-constrained decoding (Mündler et al., 2025) enforces well-typedness during generation. These methods require control over the decoding loop, which hosted APIs do not expose. *Post-hoc checking and feedback* validates complete outputs. Jigsaw (Jain et al., 2022) post-processes LLM output with program analysis and repair; Self-Debugging (Chen et al., 2024) and Reflexion (Shinn et al., 2023) feed execution or error messages back to the model.

2.5 Gap and positioning

Rule-based migration is precise but cannot synthesise; LLM migration synthesises but is not grounded; retrieval provides context but not guarantees; constrained decoding provides guarantees but needs access to the decoder. MorphAPI combines the strengths that work with any hosted model: (i) a typed program graph, in the spirit of code property graphs, that produces *verified* facts for one call site rather than retrieved text; (ii) a post-hoc gate driven by the real type checker that turns compiler diagnostics into a list of valid alternatives; and (iii) one-hop impact

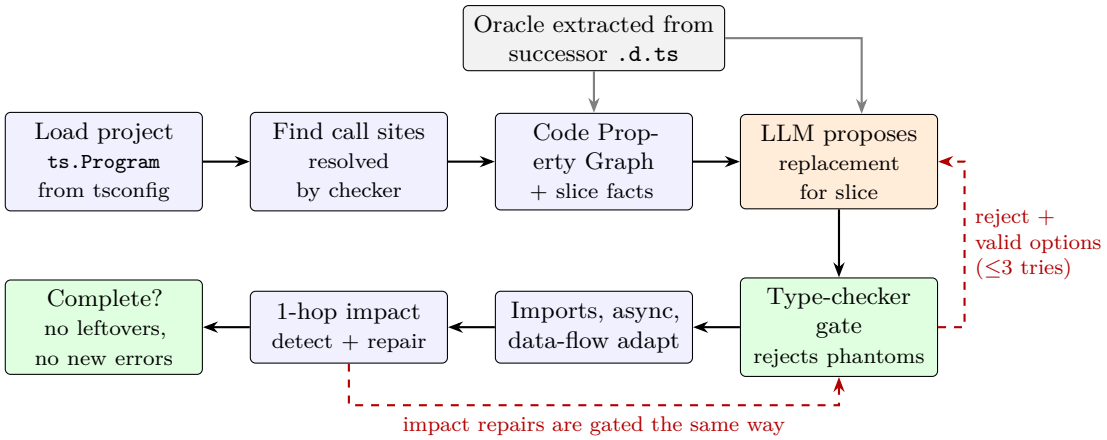


Figure 1: The generic MorphAPI pipeline. Blue stages are deterministic static analysis, orange is the LLM, green stages are judged by the TypeScript type checker. Accepted proposals continue to the bottom row; dashed arrows are feedback loops.

analysis that repairs breakage around the edit. To my knowledge, no prior migration tool combines per-call-site typed slices, compiler-grounded valid-option feedback and impact repair, and evaluates each through ablation.

3 Progress So Far

3.1 Overview

The implementation is a TypeScript monorepo: a core library (`@morphapi/core`, about 6,700 lines; the generic pipeline and graph modules account for about 3,500), eleven pairs of old/new SDK packages with client fixtures, per-scenario baselines, and a browser demo used for supervision meetings. Work proceeded in two phases.

Phase 1 (per-scenario hybrid baselines). Each scenario had a hand-written AST finder, a hand-written oracle (a JSON allow-list of SDK symbols with known phantom symbols), an inspector and a scenario-specific gate (behavioural, completeness, security or anti-cheat). This established the failure taxonomy and showed that constraining the LLM to AST spans and checking it against an oracle works: on the latest snapshot the LLM-only baselines passed 1 of 11 scenario gates and the hybrid baselines passed 11 of 11.

Phase 2 (generic pipeline). Phase 1 did not scale: every new library required a new finder, oracle and gate. Phase 2 replaced all scenario-specific code with one pipeline (Figure 1) in which a scenario is described only by its `tsconfig`, the deprecated symbol, the successor module and the prose guide. The rest of this section describes Phase 2.

3.2 Benchmark (O1)

Table 1 lists the 11 scenarios. Ten follow published failure modes of real SDK changes; the Plaid enums mirror the published Plaid TypeScript SDK, while the others use synthetic SDK packages that reproduce the same class of change in a controlled way. Each scenario has an intentionally vague migration guide, the same gap that causes hallucination in practice. In total there are 21 call sites.

Table 1: Benchmark scenarios. "Sites" is the number of call sites the pipeline must migrate.

ID	Deprecated API → successor	What goes wrong without grounding	Sites
morphpay	`ChargesApi.create` → builder	Invented `CaptureMode.Automatic`	2
plaid	string codes → `CountryCode` enum	`CountryCode.US` instead of `Us`	2
openai	`ChatCompletion.create(engine)` → `chat.completions.create(model)`	Stale `engine`, wrong client constructor	2
stripe	dollars → cents	Type-checks, wrong amount (behavioural)	2
auth	`jwt.verify(secret)` → JWKS	`@ts-ignore` / `as any` evasion	2
envelope	`User[]` → `{ data: User[] }`	Result consumers read the wrong shape	1
async	`S3.getObject` (sync) → `send(Command)`	Callers miss `await`	2
mail	`sendEmail` in several files → `send`	Some sites left on the old API	4
stripe-errors	`charges.create` + `CardError` → new hierarchy	Stale `catch` of removed class	2
discriminator	`type` → `event_type` with new values	`switch` arms on old values	1
hmac	static token → HMAC verification	Insecure compare kept (behavioural)	1

3.3 Loading the project and finding call sites

MorphAPI parses the project's `tsconfig.json` exactly as `tsc` does, so path aliases, module resolution and installed type declarations are honoured, and builds an incremental program session. Proposals are spliced into this session in memory and re-checked without writing to disk; unchanged files and their diagnostics are reused.

Call sites are found *by meaning, not by name*. The deprecated API is given as a specification such as `plaid-link-v1#PlaidApi.linkTokenCreate`; MorphAPI resolves it to a declaration through the checker and returns every call whose callee resolves to that declaration, following renamed imports, namespace imports, path aliases and NodeNext `.js` imports. A textual match on `linkTokenCreate` would miss `import { linkTokenCreate as mk }` and would wrongly flag an unrelated function with the same name. On the benchmark, the generic finder agrees with the Phase 1 hand finders on 8 of 11 scenarios; the other three hand finders target non-call sites (a `catch` clause, a `switch` and an inline token comparison).

3.4 The Code Property Graph: an AST with metadata (O2)

Why a syntax tree is not enough. A TypeScript AST is produced by the parser from the text of one file. It records node kinds and positions, for example that `CountryCode.US` is a `PropertyAccessExpression`, but nothing about meaning. To make this concrete I parsed a snippet containing `enum CountryCode { Us = "US" }` and `CountryCode.US`: the parser reported *zero* syntax errors, while the type checker reported "*Property 'US' does not exist on type 'typeof CountryCode'. Did you mean 'Us'?*". Everything a migration needs to know (which library a call belongs to, what type a value has, where it comes from, who uses the result and what replaces the old API) lives in the type checker and in analyses built on top of it. MorphAPI's graph is the AST plus these answers.

Construction. `buildCodePropertyGraph` takes the program session, the project files, the call-site spans, the deprecated symbols and the successor module, and proceeds in six steps:

1. **Program.** Reuse the session's program and checker; if the successor package's declaration file is not yet in the program, add it so successor symbols can be resolved.

Table 2: Node kinds and their metadata. Every property is produced by the TypeScript checker or by an analysis over its answers.

Node	Properties (metadata)	Source of the metadata
`function`	name, async, parameters with types, return type	declaration + `getSignatureFromDeclaration`
`variable`	name, role (param/def/assign/use), resolved type, line	occurrence analysis + `getTypeAtLocation`
`call_site`	text, resolved signature, arguments mapped to parameters with types, deprecation reason	`getResolvedSignature`, `getTypeOfSymbolAtLocation`, JSDoc tags
`api_symbol`	owning package, role (deprecated/successor), signature, declaration file, resolved flag	`getSymbolAtLocation` + alias resolution; successor module exports

Table 3: Edge kinds.

Edge	Meaning
`comesFrom`	a use reads the value of an earlier definition of the same symbol
`computedFrom`	a definition is computed from these variable occurrences
`argument_of`	a variable occurrence is passed into a call site (labelled with the parameter)
`contains`	a function contains a call site
`calls`	a function calls another project function (resolved declaration, not name)
`resolves_to`	a call site resolves to a deprecated API symbol
`migrates_to`	a deprecated API symbol is replaced by a successor symbol

2. **Functions.** Create a `function` node for each function declaration, method and arrow function bound to a constant, with `async`, parameter list and return type from the checker.

3. **Typed data flow.** Within each function, record every occurrence of a local or parameter as a `variable` node with a role (`param`, `def`, `assign`, `use`) and its checker type. A `comesFrom` edge links each use to the most recent definition of the same symbol; a `computedFrom` edge links a definition to the variables its initialiser reads.

4. **Calls.** For every call expression, resolve the callee through aliases to its declaration; if that declaration is a project function, add a `calls` edge from the enclosing function (annotated with the line and whether it is awaited).

5. **Successor symbols.** Look up each successor name in the successor module's exports through the checker and create an `api_symbol` node with its printed type or signature.

6. **Call sites.** For each span, find the target call, resolve its callee to the deprecated declaration (`resolves_to`), link that declaration to every successor (`migrates_to`), record the resolved signature, map each argument to the parameter it fills with both types, record why it is deprecated (an `@deprecated` JSDoc tag, the scenario's list, or the finder match), and find the variable the result is bound to and every later use of the *same symbol*.

Tables 2 and 3 summarise the node and edge kinds and where each property comes from.

Worked example. Figure 2 shows the graph for the first Plaid call site (16 nodes and 15 edges in total; the figure shows the part around the call at line 14). The checker labels `client`

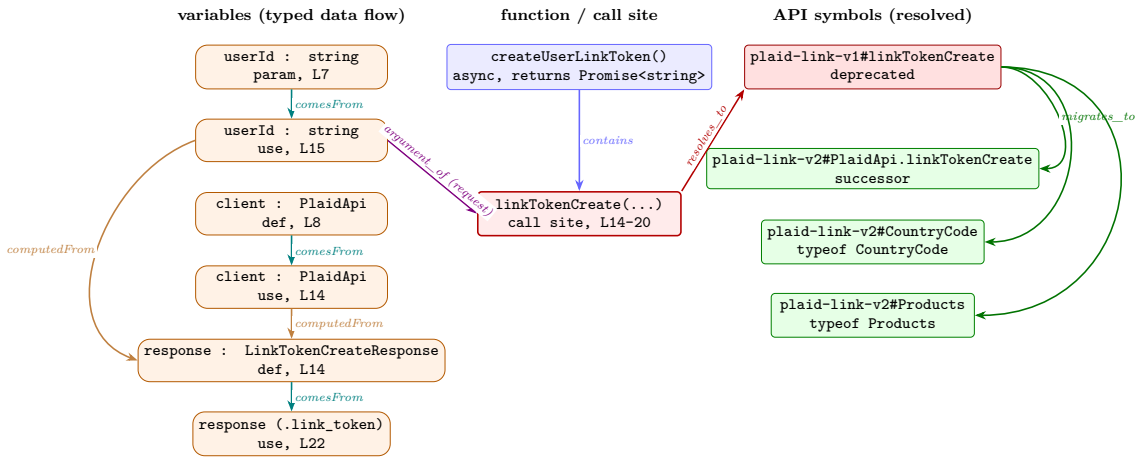


Figure 2: Part of the Code Property Graph for the Plaid call at line 14. The call site also carries its resolved signature, `(request: LinkTokenCreateRequest) => Promise<LinkTokenCreateResponse>`. Every type label and every arrow to an API symbol comes from the TypeScript checker; a plain AST contains only the node kinds and positions.

as `PlaidApi` from *plaid-link-v1*, which is how MorphAPI knows this call must be migrated; the data-flow edges show that `response` is computed from `client` and `userId` and consumed at line 22; and the `migrates_to` edges point to the three successor symbols, including `CountryCode`.

What the graph adds over a plain AST. Table 4 summarises the difference on the running example. The plain AST can locate the call and give its text; every other row needs the checker.

Graph health. Because a mis-configured project silently degrades types to `any`, the graph reports its own health: the fraction of checked calls whose callee is unresolved or collapsed to `any`, project files missing from the program, and `tsconfig` errors. Explicit `any` written by the developer is listed separately. A headless check (`npm run graph:cli`) fails if any file is missing or the collapse rate is above 0%; all 11 scenarios currently pass (Plaid: 0 of 8 unresolved).

3.5 Slice facts: what the LLM receives

For each call site, `sliceFactsFor` builds the graph focused on that site and reads a fixed set of facts off it (Listing 2). The prompt contains the *slice* (the call and its member chain, or the enclosing statement for builder and multi-step successors), these facts, the successor API surface and the guide; it never contains the whole file.

Listing 2: Slice facts for the Plaid call at line 14 (abridged).

```
{ "target_call": "client.linkTokenCreate({ ..., country_codes: [\"US\"], ... })",
  "slice": { "file": "link.ts", "lines": [14, 20] },
  "resolved_api": { "symbol": "plaid-link-v1#linkTokenCreate", "deprecated": true,
    "signature": "(request: LinkTokenCreateRequest): Promise<LinkTokenCreateResponse>" },
  "successor_api": [ { "symbol": "plaid-link-v2#PlaidApi.linkTokenCreate" },
                     { "symbol": "plaid-link-v2#CountryCode", "signature": "typeof
    CountryCode" },
                     { "symbol": "plaid-link-v2#Products",    "signature": "typeof Products"
    } ],
  "arguments": [ { "index": 0, "param": "request", "paramType": "LinkTokenCreateRequest" } ],
  "data_flow": [ { "variable": "userId", "declared_type": "string",
                   "origin": "parameter of createUserLinkToken() (line 7)" } ],
  "result": { "binding": "response", "type": "LinkTokenCreateResponse",
```

Table 4: Plain AST versus MorphAPI's Code Property Graph on the Plaid call.

Question	Plain AST	Code Property Graph
What is `client`?	an identifier	`PlaidApi` from plaid-link-v1
Is this call the deprecated API?	name match only; misses aliases	resolved declaration in plaid-link-v1
What replaces it?	—	`PlaidApi.linkTokenCreate`, `CountryCode`, `Products` with types
Where does `userId` come from?	—	parameter of `createUserLinkToken`, line 7, `string`
Who uses the result?	unrelated identifier nodes	`response.link_token` at line 22 (same symbol)
Who calls this function?	—	`calls` edges, with awaited/not awaited
Is `CountryCode.US` valid?	yes (valid grammar)	no; members are `Us`, `Gb`, `Ca`, `Fr`, `De`
Is `PLAID_SECRET` always defined?	—	`string | undefined`; successor requires `string`

```
                "downstream_usages": ["response.link_token at line 22"] },
    "enclosing_function": { "name": "createUserLinkToken", "async": true,
                            "return_type": "Promise<string>" },
    "callers_1hop": [] }
```

Each field is a graph query: `resolved_api` follows `resolves_to`; `successor_api` follows `migrates_to`; `data_flow` follows `argument_of` into the site and then `comesFrom` back to the definition, whose declaration the checker classifies as a parameter, local or import; `result` walks up from the call to its binding and collects later identifiers that resolve to the same symbol; `callers_1hop` reads the `calls` edges into the enclosing function.

Token cost. The facts are not smaller than the file on these fixtures (Plaid: file 259 tokens, slice 44, facts 505), because they add information the file does not state. The benefit claimed is precision and locality, not token savings; Section 3.9 measures it.

3.6 Oracle extraction and the type-checker gate (O3)

Oracle. Phase 1 used hand-written JSON oracles. Phase 2 extracts the oracle from the successor package's declaration file: `extractOracle` lists the module's exports through the checker, collects members of reachable types (following return types of methods and factories), and records how to obtain a value of each type (constructor or factory signature). For Plaid it finds 7 exports, including `createPlaidClient`, `CountryCode`, `Products` and `PlaidApi` (whose member is `linkTokenCreate`), and that a `PlaidApi` is obtained by calling `createPlaidClient({ clientId, secret })`. The oracle feeds the prompt, import reconciliation and the gate's suggestions.

Gate. A proposal is spliced into the in-memory program and only diagnostics inside the patched range are judged (the rest are left to impact analysis). Diagnostics are classified by TypeScript error code: missing members (TS2339, TS2551) and missing names (TS2304, TS2552) are *phantoms*; missing modules or exports (TS2305, TS2307, TS2724) are phantoms; other errors are

type errors, and for a string or number literal the gate lists the literal members of the expected union. For a missing member, the gate asks the checker for the properties of the receiver's type *at the error location* and ranks them by case-insensitive match and edit distance. It also rejects evasion (`@ts-ignore`, `as any`) and leftovers (the proposal still calls the deprecated symbol), and treats dynamic access such as `obj["x"]` as a warning. Rejections are turned into feedback and the model retries, up to three attempts; there is no template fallback.

Listing 3 shows the gate on the Plaid call from the saved gpt-4o-mini run. The first proposal used `CountryCode.US`; the gate rejected it with the valid options, and the second proposal was accepted. The same happened for `GB`→`Gb`.

Listing 3: Gate feedback on the Plaid call (saved run, gpt-4o-mini).

```
Attempt 1  country_codes: [CountryCode.US]
  REJECTED  phantom  TS2551  "Symbol US does not exist on typeof CountryCode."
            Valid options: [Us, Ca, De]
Attempt 2  country_codes: [CountryCode.Us]
  ACCEPTED
```

3.7 Follow-up steps and impact repair (O4)

After a proposal is accepted: (i) if it introduced `await` in a non-async function, the function becomes `async` and its return type `Promise<...>`; (ii) successor imports are added in the file's quote style and unused imports from the deprecated module are removed; (iii) if the result type changed (for example an envelope instead of an array), an adapter or read-path rewrite is tried and kept only if it restores the baseline diagnostics. Then **one-hop impact** compares function signatures before and after and reports `missing_await`, `same_api_leftover`, `deprecated_module_use`, `stale_import` and `new_type_error` in changed files, their callers and importers. Missing awaits are fixed deterministically; the rest go back to the LLM as statement repairs, gated the same way, and a repair is kept only if it reduces the file's error count (at most two rounds). A run is *complete* when every site was accepted, nothing resolves to or is imported from the deprecated API, and there are no diagnostics beyond the baseline.

3.8 Design alternatives investigated

- **Whole-file prompting vs. slices.** The LLM-only baseline gives the model the whole file. It is cheaper in tokens but rewrites unrelated code (Section 3.9). Slices with facts were chosen for locality.
- **Hand-written oracles vs. extracted oracles.** Phase 1 oracles listed allowed symbols and known phantoms (e.g. `CountryCode.US`). They worked but had to be written per library and could go stale. Phase 2 reads the declarations and lets the checker judge, which removes per-library authoring.
- **Template fallback.** An early version substituted a hand-written correct answer when the model failed every attempt. This hid failures and was removed; a site that fails all attempts is now reported as not migrated.
- **Learned graph embeddings (GraphCodeBERT).** I prototyped a GraphCodeBERT (Guo et al., 2021) service that embedded functions with their data-flow graph and ranked code similar to a deprecated call. It found semantically similar code, but embeddings give similarity scores, not the yes/no answer the gate needs ("does `CountryCode.US` exist?"), and it added a Python and GPU dependency. It was set aside in favour of the type checker; it may return for finding non-call sites that do not mention the deprecated API (Section 4).

Table 5: Pipeline results on 11 scenarios / 21 call sites (one run per arm; gpt-4o-mini unless stated).

Arm	Complete	Phantoms rejected	Churn	Prompt tokens	Incomplete scenarios
MorphAPI (all)	**11/11**	10	**0%**	43,321	—
MorphAPI, Claude Sonnet 4.5	11/11	4	0%	41,637	—
LLM-only (whole file)	5/11	–	67%	6,267	morphpay, plaid, auth, envelope, mail, discriminator
No slices (file + gate + impact)	10/11	12	0%	42,049	stripe-errors
No gate / feedback	7/11	–	0%	40,894	async, morphpay, openai, stripe-errors
No impact repair	6/11	9	0%	34,812	discriminator, mail, morphpay, openai, stripe-errors

- **Pure AST recipes.** Four scenarios have hand-written AST-only pilots. They migrate what the recipe enumerates and stop: e.g. in *async* the leaf is migrated but the caller's missing `await` is detected and left unfixed; in *mail* two of four sites remain. This confirms the expected ceiling of rule-based migration.

3.9 Preliminary evaluation (O5)

Setup. All arms run the same code path with mechanisms switched off: **LLM-only** (whole file, one rewrite, no gate, no impact); **No slices** (whole file instead of slice facts, with gate and impact); **No gate** (gate findings recorded but never rejected, one attempt); **No impact** (no impact repair). The model is gpt-4o-mini at temperature 0.2; the full pipeline was also run with Claude Sonnet 4.5. "Churn" is the share of changed lines that belong to no migrated site, impact repair, import or deterministic follow-up.

Results. Table 5 summarises the runs. The full pipeline completed all 11 scenarios with both models and changed no unrelated lines; the gpt-4o-mini result was the same on three consecutive full runs. The LLM-only baseline completed 5, and 155 of its 232 changed lines (67%) were unrelated to the migration. On Plaid it produced `CountryCode.US` and `CountryCode.GB` and ended with two TS2551 errors.

What the ablations show. Removing impact repair costs the most (6/11): breakage one hop away (callers that must now await, catch clauses on removed error classes, switch arms on old values) is never repaired. Removing the gate costs 4 scenarios: phantoms are accepted without feedback. On Plaid, the no-gate run still completed because impact repair, which also uses the checker, later caught the new type error and fixed `US`→`Us`; both safety nets depend on type information. Replacing slice facts with the whole file costs only one scenario (stripe-errors, where the model kept calling `paymentIntents` on the old client), so the main value of slices here is locality rather than completion. Running the first Claude experiment also exposed two generic weaknesses that were fixed for both models: literal type errors now list the allowed literal values, and a stray pair of backticks around a one-line answer (which TypeScript parses as a template string) is stripped.

Threats to validity. (i) One run per arm; LLMs are non-deterministic, so these are snapshot results, not rates. (ii) The fixtures are small (one to three files) and mostly synthetic; results may not transfer to large real projects. (iii) Completion is a type-level judgement: the pipeline does

not yet run the behavioural checks (e.g. the unit conversion in *stripe*) that the Phase 1 baselines had. (iv) Impact analysis is one hop. (v) The successor symbol list per scenario is configured, not discovered; the generic finder handles call sites only, so the inline token comparison in *hmac* is not migrated. (vi) The extracted oracle's member lists for enums currently record string-prototype members rather than the enum members; the gate is not affected because it queries the checker at the error location, but this should be fixed.

3.10 Tooling for supervision

A browser demo (`npm run demo:ui`) shows, for any scenario, the source with the call site highlighted, the typed syntax tree, the typed data-flow graph, the full Code Property Graph with clickable nodes, the slice facts with token counts, the one-hop impact and graph health, and the pipeline run with every gate attempt and the final diff. Runs can be replayed from saved JSON or started live, which makes the mechanism inspectable during meetings.

4 Research Plan for Next Semester

4.1 Research questions

RQ1 How much does type-grounded context and feedback improve migration completeness and locality over LLM-only migration, measured over repeated runs and several models?

RQ2 Which mechanism (slice facts, type-checker gate, impact repair) contributes most, and does the answer change with model strength?

RQ3 Does the approach transfer from controlled fixtures to real migrations in open-source TypeScript projects?

RQ4 How often does a type-correct migration still change behaviour, and can behavioural checks be integrated into the generic pipeline?

4.2 Planned work

1. **Statistical evaluation.** Run every arm at least five times per scenario with gpt-4o-mini, Claude and one open-weight model; report completion rates with confidence intervals, phantom counts, attempts, churn and cost.

2. **Real-world benchmark.** Mine 15–25 real migrations from open-source TypeScript repositories (for example major-version upgrades of the OpenAI, Stripe and AWS SDKs), use the developers' commits as references, and compare with the pipeline's output.

3. **Behavioural gates in the pipeline.** Port the Phase 1 behavioural, security and completeness checks into the generic pipeline as optional test hooks, so that "complete" can also mean "tests pass".

4. **Graph improvements.** Extend impact from one hop to a bounded transitive closure; fix enum member extraction in the oracle; discover successor symbols automatically from the declarations and the guide instead of configuring them; and reach non-call sites (catch clauses, switches, inline comparisons) through type relationships, possibly reusing learned embeddings for candidate ranking.

5. **Baselines.** Add a codemod baseline (jscodeshift recipes) for the real-world set, and compare with a retrieval-only context (similar code from the repository instead of graph facts).

6. **Write-up.** Final report, demo and presentation.

4.3 Timeline

Table 6: Plan for Semester 2 (weeks of the NUS academic calendar).

Weeks	Work	Deliverable
Break	Fix oracle enum extraction; repeated-run harness; add open-weight model	multi-seed runs on current benchmark
1–3	Mine and set up real-world migrations; codemod baseline	real-world benchmark (15–25 cases)
4–6	Behavioural/test hooks in pipeline; transitive impact; successor discovery	pipeline v2
Recess	Full experiment matrix (arms × models × seeds × benchmarks)	results tables
7–9	Analysis: RQ1–RQ4, error taxonomy of failures	draft evaluation chapter
10–11	Final report writing; demo polish	full draft to supervisor
12–13	Revisions, presentation	final report and presentation

4.4 Risks

Real projects may not compile cleanly before migration, which weakens a type-checker gate; the pipeline already compares against baseline diagnostics, and projects that cannot build will be excluded and reported. API costs for repeated runs are bounded by using a small model for most runs and the larger model for a subset. If behavioural tests are unavailable for real projects, developer commits will serve as the reference.

5 Conclusion

This semester established the problem, the benchmark and a working generic pipeline. The main technical contribution so far is a Code Property Graph built on the real TypeScript program, which turns a syntax tree into verified facts about one call site (types, resolved declarations, data flow, callers, result consumers and successor symbols), together with a type-checker gate that turns compiler diagnostics into valid-option feedback. On 11 scenarios, the pipeline completed every migration with zero unrelated edits where an LLM-only baseline completed five, and ablations indicate that the gate and impact repair are the critical parts. Next semester will test whether these results hold over repeated runs, other models and real open-source migrations.

References

Lakshya A. Agrawal, Aditya Kanade, Navin Goyal, Shuvendu K. Lahiri, and Sriram K. Rajamani. Monitor-guided decoding of code LMs with static analysis of repository context. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Miltiadis Allamanis, Marc Brockschmidt, and Mahmoud Khademi. Learning to represent programs with graphs. In *International Conference on Learning Representations (ICLR)*, 2018.

Aylton Almeida, Laerte Xavier, and Marco Tulio Valente. Automatic library migration using large language models: First results. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2024.

Gavin Bierman, Martín Abadi, and Mads Torgersen. Understanding TypeScript. In *ECOOP 2014 – Object-Oriented Programming*, volume 8586 of *Lecture Notes in Computer Science*, pages 257–281. Springer, 2014.

Christopher Bogart, Christian Kästner, James Herbsleb, and Ferdian Thung. How to break an API: Cost negotiation and community values in three software ecosystems. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 109–120, 2016.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. arXiv:2107.03374, 2021.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.

Danny Dig and Ralph Johnson. How do APIs evolve? A story of refactoring. *Journal of Software Maintenance and Evolution: Research and Practice*, 18(2):83–107, 2006.

Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, 2020.

Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems*, 9(3): 319–349, 1987.

Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. GraphCodeBERT: Pre-training code representations with data flow. In *International Conference on Learning Representations (ICLR)*, 2021.

Johannes Henkel and Amer Diwan. CatchUp!: Capturing and replaying refactorings to support API evolution. In *Proceedings of the 27th International Conference on Software Engineering (ICSE)*, pages 274–283, 2005.

Naman Jain, Skanda Vaidyanath, Arun Iyer, Nagarajan Natarajan, Suresh Parthasarathy, Sriram Rajamani, and Rahul Sharma. Jigsaw: Large language models meet program synthesis. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, pages 1219–1231, 2022.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Ameya Ketkar, Oleg Smirnov, Nikolaos Tsantalis, Danny Dig, and Timofey Bryksin. Inferring and applying type changes. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, 2022.

Raula Gaikovina Kula, Daniel M. German, Ali Ouni, Takashi Ishio, and Katsuro Inoue. Do developers update their library dependencies? An empirical study on the impact of security advisories on library migration. *Empirical Software Engineering*, 23(1):384–417, 2018.

Maxime Lamothe, Yann-Gaël Guéhéneuc, and Weiyi Shang. A systematic review of API evolution literature. *ACM Transactions on Software Engineering and Methodology*, 30(4), 2021.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, Li Zhang, Zhongqi Li, and Yuchi Ma. Exploring and evaluating hallucinations in LLM-powered code generation. arXiv:2404.00971, 2024.

Meta Open Source. jscodeshift: A JavaScript codemod toolkit. `https://github.com/facebook/jscodeshift`, 2026. Accessed October 2026.

Microsoft. Using the TypeScript compiler API. `https://github.com/microsoft/TypeScript/wiki/Using-the-Compiler-API`, 2026. Accessed October 2026.

Moderne, Inc. OpenRewrite: Automated mass refactoring of source code. `https://docs.openrewrite.org`, 2026. Accessed October 2026.

Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, and Martin Vechev. Type-constrained code generation with language models. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2025.

Hoan Anh Nguyen, Tung Thanh Nguyen, Gary Wilson, Jr., Anh Tuan Nguyen, Miryung Kim, and Tien N. Nguyen. A graph-based approach to API usage adaptation. In *Proceedings of the ACM International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, pages 302–321, 2010.

Stoyan Nikolov, Daniele Codecasa, Anna Sjovall, Maxim Tabachnyk, Satish Chandra, Siddharth Taneja, and Celal Ziftci. How is Google using AI for internal code migrations? arXiv:2501.06972, 2025.

Romain Robbes, Mircea Lungu, and David Röthlisberger. How do developers react to API deprecation? The case of a Smalltalk ecosystem. In *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering (FSE)*, 2012.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Joseph Spracklen, Raveen Wijewickrama, A. H. M. Nazmus Sakib, Anindya Maiti, Bimal Viswanath, and Murtuza Jadliwala. We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs. In *34th USENIX Security Symposium*, 2025.

Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. Copiloting the copilots: Fusing large language models with completion engines for automated program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 172–184, 2023.

Mark Weiser. Program slicing. In *Proceedings of the 5th International Conference on Software Engineering (ICSE)*, pages 439–449, 1981.

Laerte Xavier, Aline Brito, André Hora, and Marco Tulio Valente. Historical and impact analysis of API breaking changes: A large-scale study. In *IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 138–147, 2017.

Fabian Yamaguchi, Nico Golde, Daniel Arp, and Konrad Rieck. Modeling and discovering vulnerabilities with code property graphs. In *2014 IEEE Symposium on Security and Privacy (S&P)*, pages 590–604, 2014. doi: 10.1109/SP.2014.44.

Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.